



+



Spring Boot et Thymeleaf

1 QU'EST-CE QUE THYMELEAF ?

Thymeleaf est un moteur de template (**Template Engine**) **Java XML/XHTML/HTML5** qui peut travailler à la fois dans des environnements Web (Servlet) et non Web.

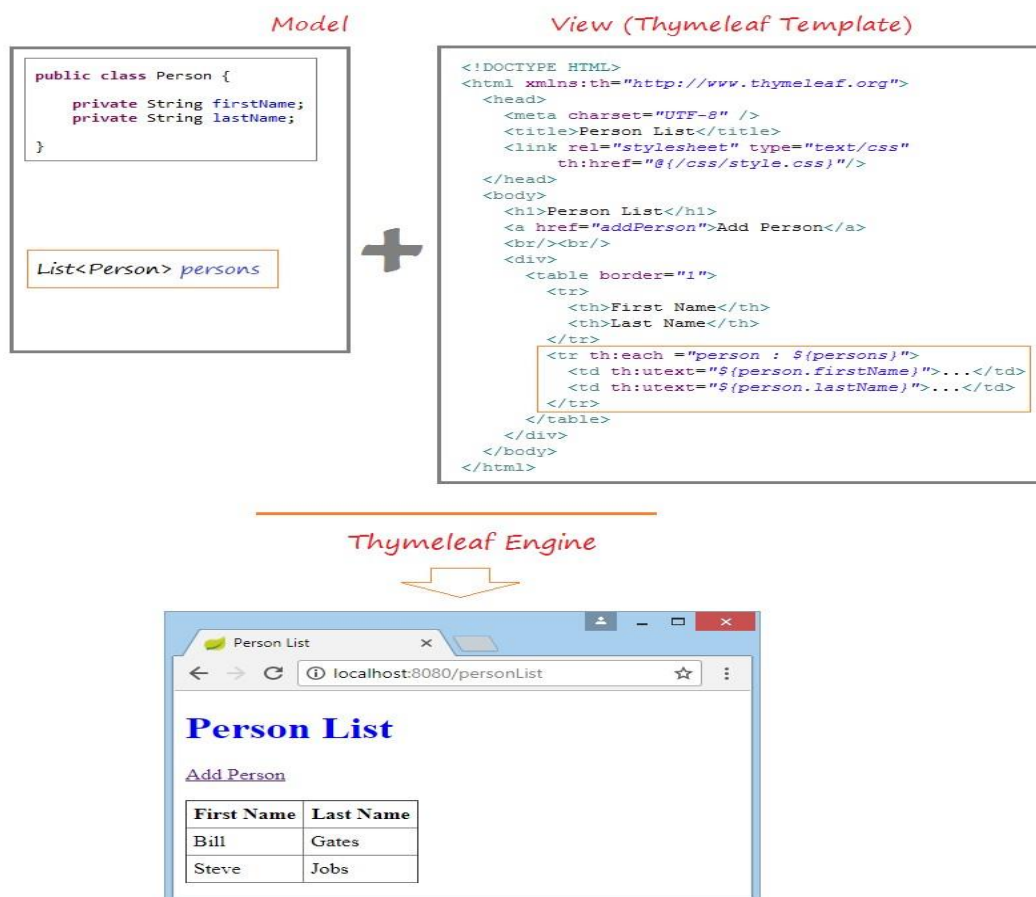
Il est mieux adapté pour diffuser **XHTML/HTML5** sur **View** (View Layer) des applications **Web** basées sur **MVC**.

Mais il peut traiter n'importe quel fichier **XML** même dans des environnements hors ligne (offline).

Il fournit une intégration complète de **Spring Framework**.

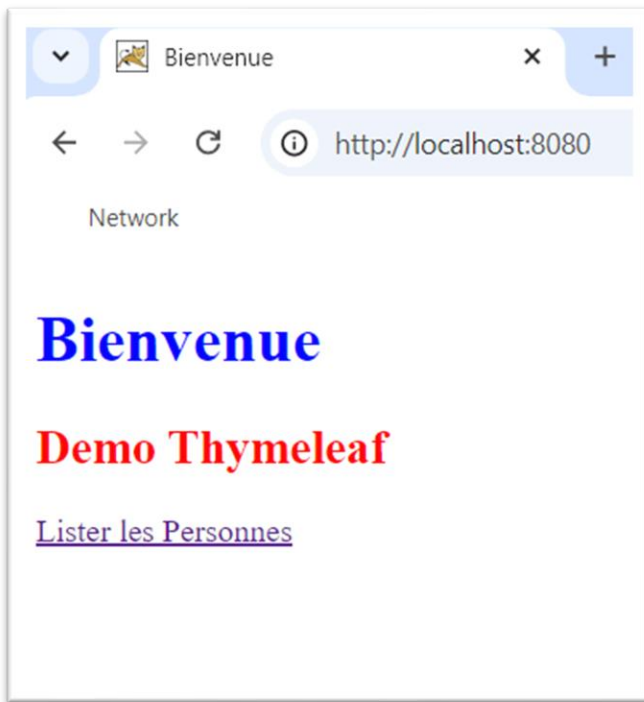
Le fichier modèle (Template file) de **Thymeleaf** est un fichier au format **XML/XHTML/HTML5**.

Thymeleaf Engine (le moteur Thymeleaf) va lire un fichier modèle et le combiner avec des objets **Java** pour générer (generate) un autre document.



Thymeleaf peut être utilisé pour remplacer **JSP** sur la couche **View** (View Layer) de l'application **Web MVC**. **Thymeleaf** est un logiciel à code source ouvert (open source) sous licence **Apache 2.0**.

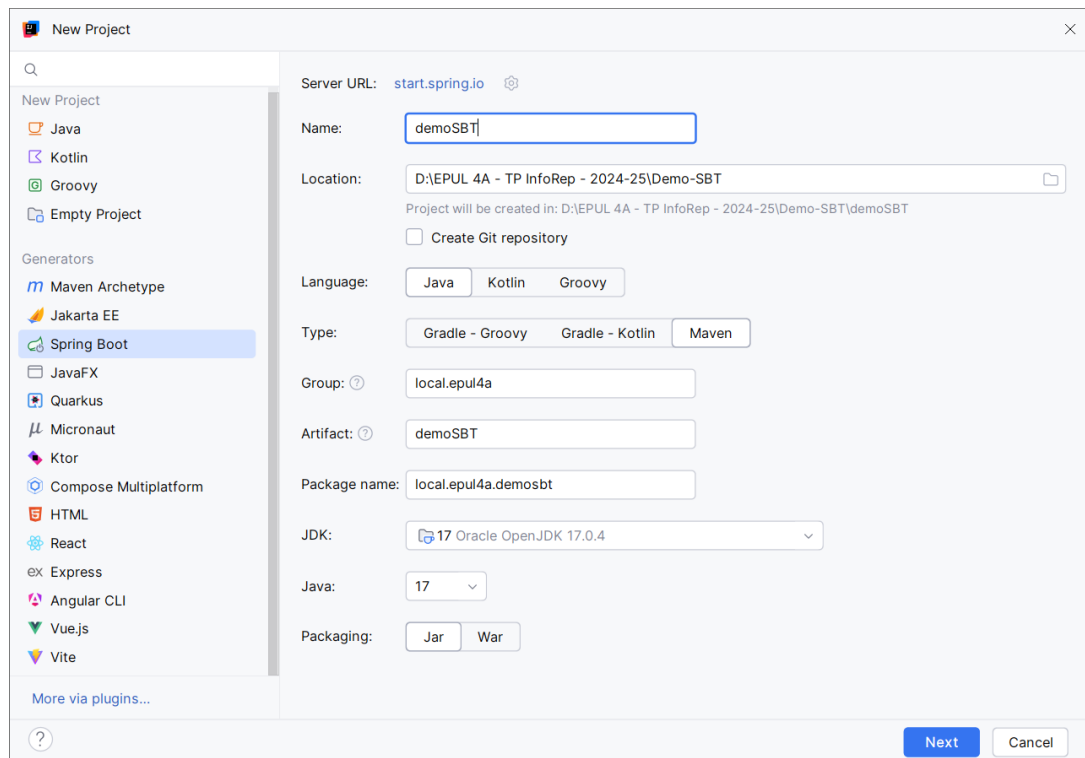
Dans cette présentation rapide, je vous propose de réaliser l'application dont voici la page d'accueil :



2 CREER L'APPLICATION

Utilisation de l'IDE IntelliJ IDEA de l'éditeur JetBrains.

Lancer L'ide et choisir nouveau projet.



Saisissez :

- **Name:** demoSBT
- **Group:** local.epul4a
- **Artifact:** demoSBT
- **Description:** Spring Boot and Thymeleaf
- **Package:** local.epul4a.demosbt

Sélectionnez 2 technologies **Web** et **Thymeleaf**.

New Project

Spring Boot: 3.3.5

Dependencies:

Search

- ☐ GraphQL DGS Code Generation
- ☒ Spring Boot DevTools
- ☐ Lombok
- ☐ Spring Configuration Processor
- ☐ Docker Compose Support
- ☐ Spring Modulith

Web

- ☒ Spring Web
- ☐ Spring Reactive Web
- ☐ Spring for GraphQL
- ☐ Rest Repositories
- ☐ Spring Session
- ☐ Rest Repositories HAL Explorer
- ☐ Spring HATEOAS
- ☐ Spring Web Services
- ☐ Jersey
- ☐ Vaadin
- ☐ Netflix DGS
- ☐ htmx

Template Engines

- ☒ Thymeleaf
- ☐ Apache Freemarker

Thymeleaf

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

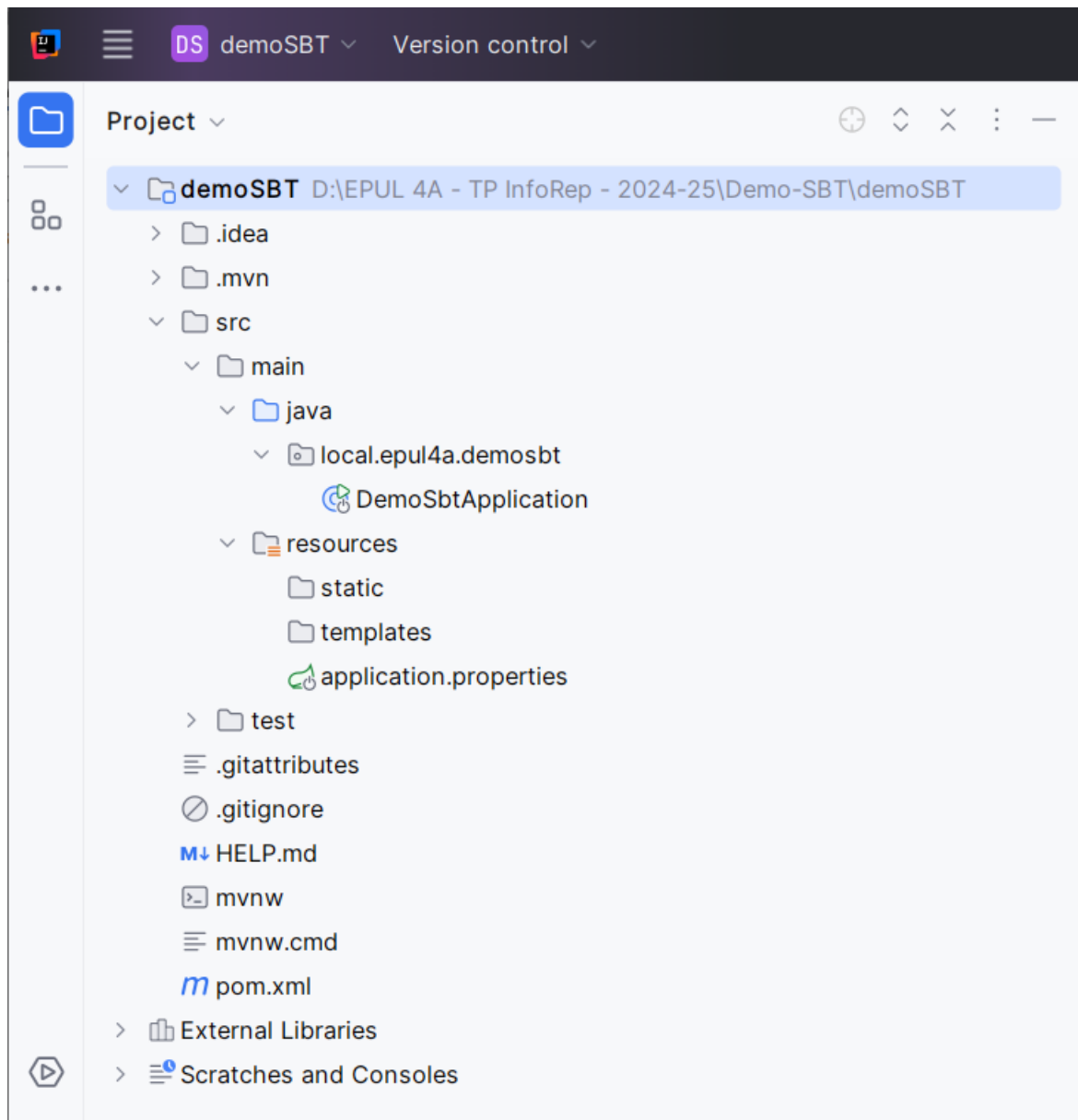
[Guide](#)

Added dependencies:

- Spring Boot DevTools
- Spring Web
- Thymeleaf

Previous Create Cancel

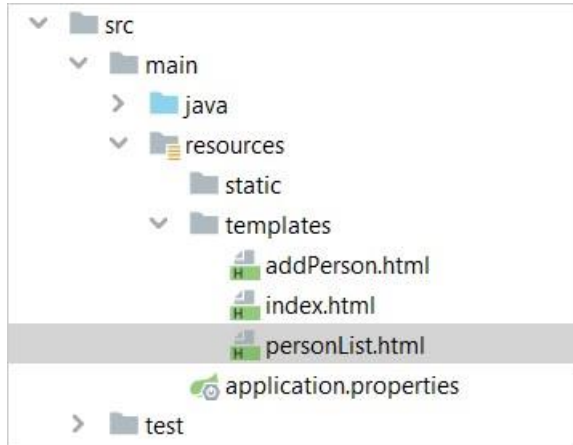
Votre projet a été créé :



3 THYMELEAF TEMPLATE

Thymeleaf Template est un fichier modèle.

Son contenu est au format **XML/XHTML/HTML5**. Nous allons créer 3 fichiers et les placer dans le répertoire **src/main/resources/templates** :



3.1 CONTENU DU FICHIER INDEX.HTML

```
<!DOCTYPE HTML>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Bienvenue</title>
<link rel="stylesheet" type="text/css" th:href="@{/css/style.css}"/>
</head>
<body>
<h1>Bienvenue</h1>
<!--/*@thymesVar id="message" type="java"*/-->
<h2 th:utext="${message}">...!..</h2>
<a th:href="@{/personList}">Listez les Personnes</a>
</body>
</html>
```

3.2 CONTENU DU FICHIER PERSONLIST.HTML

```
<!DOCTYPE HTML>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8" />
    <title>Liste des Personnes</title>
    <link rel="stylesheet" type="text/css" th:href="@{/css/style.css}"/>
  </head>
  <body>
    <style>
      table {
        border:1px solid black;
      }
    </style>
    <h1>Liste des Personnes</h1>
    <a href="addPerson">Add Person</a>
    <br/><br/>
    <div>
      <table>
        <tr>
          <th>Prénom</th>
          <th>Nom</th>
        </tr>
        <tr th:each = "person : ${persons}">
          <td th:utext="${person.firstName}">...</td>
          <td th:utext="${person.lastName}">...</td>
        </tr>
      </table>
    </div>
  </body>
</html>
```

3.3 CONTENU DU FICHIER ADDPERSON.HTML

```
<!DOCTYPE HTML>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8" />
    <title>Add Person</title>
    <link rel="stylesheet" type="text/css" th:href="@{/css/style.css}"/>
  </head>
  <body>
    <h1>Create a Person:</h1>

    <form th:action="@{/addPerson}"          th:object="${personForm}"
method="POST">
      Prénom:
      <input type="text" th:field="*{firstName}" />
      <br/>
      Last Name:
      <input type="text" th:field="*{lastName}" />
      <br/>
      <input type="submit" value="Create" />
    </form>
```

```
<br/>

<!-- Check if errorMessage is not null and not empty -->
<div th:if="${errorMessage}" th:utext="${errorMessage}"
style="color:red;font-style:italic;">
    ...
</div>

</body>
</html>
```

Vous remarquerez que les fichiers créés ci-dessus respectent les normes de **XML**, toutes les étiquettes (tag) doivent être ouvertes et fermées. Par exemple :

```
<div>A div tag</div>

<br />

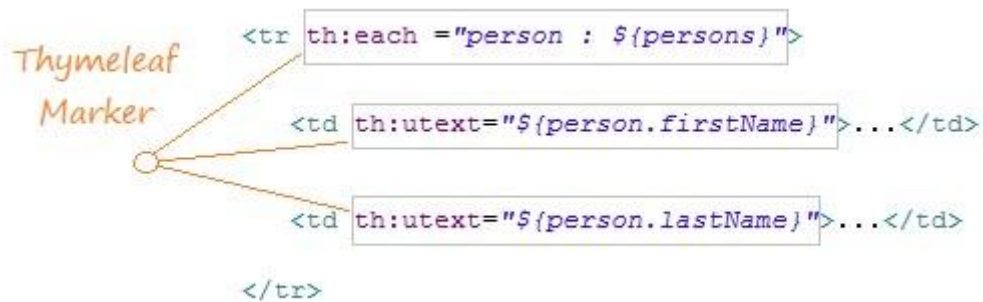
<meta charset="UTF-8" />
```

Tous les fichiers **HTML** doivent déclarer l'utilisation de **Thymeleaf Namespace**:

```
<!-- Thymeleaf Namespace -->
<html xmlns:th="http://www.thymeleaf.org">
```

Dans les fichiers modèles (Template file), il y a des **Thymeleaf Marker** (des marqueurs de Thymeleaf) qui sont des instructions aidant les données de processus de **Thymeleaf Engine**.

Thymeleaf Engine analyse les fichiers modèles (Template file) et les combine avec les données Java pour générer (generate) un nouveau document.



Ci-dessus sont des exemples qui utilisent **Context-Path** dans **Thymeleaf**:

<!-- Example 1: -->

```
<a th:href="@{/mypath/abc.html}">A Link</a>
```

Output: ==>

```
<a href="/my-context-path/mypath/abc.html">A Link</a>
```

<!-- Example 2: -->

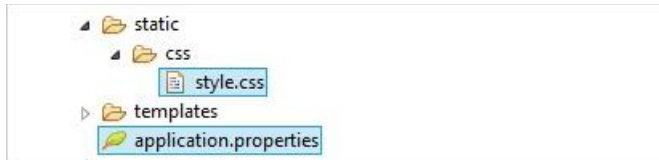
```
<form th:action="@{/mypath/abc.html}"
      th:object="${personForm}" method="POST">
```

Output: ==>

```
<form action="/my-context-path/mypath/abc.html" method="POST">
```

4 STATIC RESOURCE & PROPERTIES FILE

Pour les ressources statiques (Static Resource), par exemple : des fichiers **css**, **javascript**, **image**, ... vous devez les placer dans le dossier **src/main/resources/static** ou dans ses sous-dossiers.



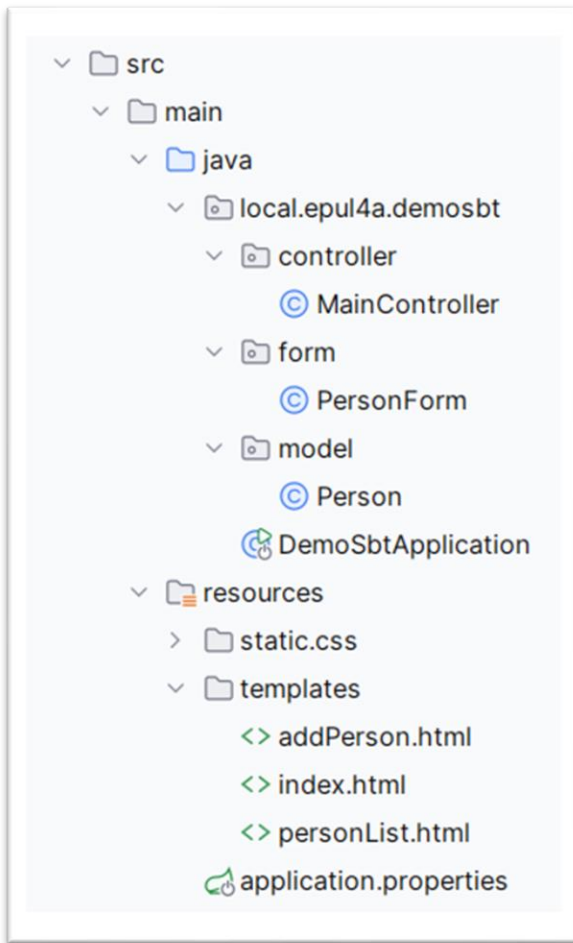
4.1 CONTENU DU FICHIER STYLE.CSS

```
h1 {  
    color:#0000FF;  
}  
h2 {  
    color:#FF0000;  
}  
table {  
    border-collapse: collapse;  
}  
table th, table td {  
    padding: 5px;  
}
```

4.2 CONTENU DU FICHIER APPLICATION.PROPERTIES

```
spring.application.name=demoSBT  
  
spring.thymeleaf.cache=false  
  
welcome.message=Demo Thymeleaf  
error.message=Prénom & Nom sont obligatoire!
```

5 MODEL, FORM, CONTROLLER CLASSES



5.1 CONTENU DU FICHIER PERSON.JAVA

```
package local.epul4a.demosbt.model;

public class Person {

    private String firstName;
    private String lastName;

    public Person() {

    }

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
}
```

```

    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}

```

5.2 LA CLASSE PERSONFORM

Elle représente des données de **FORM** lorsque vous créez une nouvelle **Personne** sur la page **addPerson**.

5.2.1 Contenu du fichier PersonForm.java

```

package org.epul.thymeleaf.form;

public class PersonForm {
    private String firstName;
    private String lastName;

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}

```

5.3 LA CLASSE MAINCONTROLLER

MainController est une classe **Controller**. Il traite les demandes d'un utilisateur et contrôle le flux (flow) de l'application.

5.3.1 Contenu du fichier MainController.java

```
package local.epul4a.demosbt.controller;

import java.util.ArrayList;
import java.util.List;

import local.epul4a.demosbt.form.PersonForm;
import local.epul4a.demosbt.model.Person;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class MainController {

    private static List<Person> persons = new ArrayList<Person>();

    static {
        persons.add(new Person("Bill", "Gates"));
        persons.add(new Person("Steve", "Jobs"));
    }

    // Injectez (inject) via application.properties.
    @Value("${welcome.message}")
    private String message;

    @Value("${error.message}")
    private String errorMessage;

    @RequestMapping(value = { "/", "/index" }, method = RequestMethod.GET)
    public String index(Model model) {

        model.addAttribute("message", message);

        return "index";
    }

    @RequestMapping(value = { "/personList" }, method = RequestMethod.GET)
    public String personList(Model model) {

        model.addAttribute("persons", persons);

        return "personList";
    }

    @RequestMapping(value = { "/addPerson" }, method = RequestMethod.GET)
    public String showAddPersonPage(Model model) {

        PersonForm personForm = new PersonForm();
        model.addAttribute("personForm", personForm);
    }
}
```

```

        return "addPerson";
    }

    @RequestMapping(value = { "/addPerson" }, method = RequestMethod.POST)
    public String savePerson(Model model, //
        @ModelAttribute("personForm") PersonForm personForm) {

        String firstName = personForm.getFirstName();
        String lastName = personForm.getLastName();

        if (firstName != null && firstName.length() > 0 //
            && lastName != null && lastName.length() > 0) {
            Person newPerson = new Person(firstName, lastName);
            persons.add(newPerson);

            return "redirect:/personList";
        }

        model.addAttribute("errorMessage", errorMessage);
        return "addPerson";
    }
}

```

6 EXECUTER DE L'APPLICATION

Exécutez **URL** ci-dessous sur le navigateur :

